Volume 03 - Book 03.10 Model Ecosystem

Version v12 draft

README.md

Book 03.10 - Model Ecosystem

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-README

Purpose

Define the Model Ecosystem blueprint package and its subsystem decomposition as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for the Model Ecosystem blueprint package and its subsystem decomposition. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for the Model Ecosystem blueprint package and its subsystem decomposition.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for the Model Ecosystem blueprint package and its subsystem decomposition.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.

- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.
- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ModelEcosystemDefined
- ModelEcosystemRegistered
- ModelEcosystemStateChanged
- ModelEcosystemReviewRequested
- ModelEcosystemGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.

- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.
- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Model Ecosystem has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

INDEX.md

Book 03.10 - Model Ecosystem

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-INDEX

Purpose

Define the Model Ecosystem blueprint package and its subsystem decomposition as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for the Model Ecosystem blueprint package and its subsystem decomposition. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for the Model Ecosystem blueprint package and its subsystem decomposition.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for the Model Ecosystem blueprint package and its subsystem decomposition.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.
- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ModelEcosystemDefined
- ModelEcosystemRegistered
- ModelEcosystemStateChanged
- ModelEcosystemReviewRequested
- ModelEcosystemGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.

- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.
- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Model Ecosystem has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

Book 03.10.01 - Model Ecosystem Overview

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-001

Purpose

Define Model Ecosystem Overview within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Model Ecosystem Overview within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Model Ecosystem Overview within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Model Ecosystem Overview within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.
- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ModelEcosystemOverviewDefined
- ModelEcosystemOverviewRegistered
- ModelEcosystemOverviewStateChanged
- ModelEcosystemOverviewReviewRequested
- ModelEcosystemOverviewGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.

- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.
- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Model Ecosystem Overview has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

book-03-10-02-model-registry.md

Book 03.10.02 - Model Registry

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-002

Purpose

Define Model Registry within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Model Registry within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Model Registry within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Model Registry within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ModelRegistryDefined
- ModelRegistryRegistered
- ModelRegistryStateChanged
- ModelRegistryReviewRequested
- ModelRegistryGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Model Registry has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

book-03-10-03-model-router.md

Book 03.10.03 - Model Router

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-003

Purpose

Define Model Router within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Model Router within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Model Router within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Model Router within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ModelRouterDefined
- ModelRouterRegistered
- ModelRouterStateChanged
- ModelRouterReviewRequested
- ModelRouterGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Model Router has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

Book 03.10.04 - Gguf Models

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-004

Purpose

Define Gguf Models within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Gguf Models within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Gguf Models within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Gguf Models within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- GgufModelsDefined
- GgufModelsRegistered
- GgufModelsStateChanged
- GgufModelsReviewRequested
- GgufModelsGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Gguf Models has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

Book 03.10.05 - Onnx Models

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-005

Purpose

Define Onnx Models within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Onnx Models within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Onnx Models within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Onnx Models within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- OnnxModelsDefined
- OnnxModelsRegistered
- OnnxModelsStateChanged
- OnnxModelsReviewRequested
- OnnxModelsGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Onnx Models has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

Book 03.10.06 - Safetensors Models

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-006

Purpose

Define Safetensors Models within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Safetensors Models within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Safetensors Models within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Safetensors Models within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- SafetensorsModelsDefined
- SafetensorsModelsRegistered
- SafetensorsModelsStateChanged
- SafetensorsModelsReviewRequested
- SafetensorsModelsGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Safetensors Models has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

book-03-10-07-transformers-models.md

Book 03.10.07 - Transformers Models

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-007

Purpose

Define Transformers Models within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Transformers Models within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Transformers Models within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Transformers Models within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- TransformersModelsDefined
- TransformersModelsRegistered
- TransformersModelsStateChanged
- TransformersModelsReviewRequested
- TransformersModelsGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Transformers Models has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

Book 03.10.08 - Pytorch Models

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-008

Purpose

Define Pytorch Models within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Pytorch Models within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Pytorch Models within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Pytorch Models within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- PytorchModelsDefined
- PytorchModelsRegistered
- PytorchModelsStateChanged
- PytorchModelsReviewRequested
- PytorchModelsGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Pytorch Models has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

Book 03.10.09 - Llama Cpp Runtime

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-009

Purpose

Define Llama Cpp Runtime within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Llama Cpp Runtime within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Llama Cpp Runtime within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Llama Cpp Runtime within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- LlamaCppRuntimeDefined
- LlamaCppRuntimeRegistered
- LlamaCppRuntimeStateChanged
- LlamaCppRuntimeReviewRequested
- LlamaCppRuntimeGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Llama Cpp Runtime has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

book-03-10-10-vllm-runtime.md

Book 03.10.10 - Vllm Runtime

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-010

Purpose

Define Vllm Runtime within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Vllm Runtime within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Vllm Runtime within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Vllm Runtime within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- VllmRuntimeDefined
- VllmRuntimeRegistered
- VllmRuntimeStateChanged
- VllmRuntimeReviewRequested
- VllmRuntimeGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Vllm Runtime has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

book-03-10-11-ktransformers-runtime.md

Book 03.10.11 - Ktransformers Runtime

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-011

Purpose

Define Ktransformers Runtime within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Ktransformers Runtime within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Ktransformers Runtime within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Ktransformers Runtime within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- KtransformersRuntimeDefined
- KtransformersRuntimeRegistered
- KtransformersRuntimeStateChanged
- KtransformersRuntimeReviewRequested
- KtransformersRuntimeGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Ktransformers Runtime has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

book-03-10-12-provider-abstraction.md

Book 03.10.12 - Provider Abstraction

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-012

Purpose

Define Provider Abstraction within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Provider Abstraction within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Provider Abstraction within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Provider Abstraction within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ProviderAbstractionDefined
- ProviderAbstractionRegistered
- ProviderAbstractionStateChanged
- ProviderAbstractionReviewRequested
- ProviderAbstractionGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Provider Abstraction has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

book-03-10-13-model-evaluation.md

Book 03.10.13 - Model Evaluation

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-013

Purpose

Define Model Evaluation within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Model Evaluation within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Model Evaluation within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Model Evaluation within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ModelEvaluationDefined
- ModelEvaluationRegistered
- ModelEvaluationStateChanged
- ModelEvaluationReviewRequested
- ModelEvaluationGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Model Evaluation has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

book-03-10-14-model-fallback.md

Book 03.10.14 - Model Fallback

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-014

Purpose

Define Model Fallback within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Model Fallback within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Model Fallback within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Model Fallback within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ModelFallbackDefined
- ModelFallbackRegistered
- ModelFallbackStateChanged
- ModelFallbackReviewRequested
- ModelFallbackGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Model Fallback has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

book-03-10-15-model-cost-control.md

Book 03.10.15 - Model Cost Control

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-015

Purpose

Define Model Cost Control within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Model Cost Control within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Model Cost Control within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Model Cost Control within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ModelCostControlDefined
- ModelCostControlRegistered
- ModelCostControlStateChanged
- ModelCostControlReviewRequested
- ModelCostControlGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Model Cost Control has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

book-03-10-16-model-events.md

Book 03.10.16 - Model Events

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-016

Purpose

Define Model Events within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Model Events within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Model Events within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Model Events within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ModelEventsDefined
- ModelEventsRegistered
- ModelEventsStateChanged
- ModelEventsReviewRequested
- ModelEventsGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Model Events has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

book-03-10-17-model-storage.md

Book 03.10.17 - Model Storage

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-017

Purpose

Define Model Storage within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Model Storage within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Model Storage within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Model Storage within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ModelStorageDefined
- ModelStorageRegistered
- ModelStorageStateChanged
- ModelStorageReviewRequested
- ModelStorageGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Model Storage has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

book-03-10-18-model-security.md

Book 03.10.18 - Model Security

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-018

Purpose

Define Model Security within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Model Security within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Model Security within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Model Security within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ModelSecurityDefined
- ModelSecurityRegistered
- ModelSecurityStateChanged
- ModelSecurityReviewRequested
- ModelSecurityGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Model Security has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

book-03-10-19-model-testing.md

Book 03.10.19 - Model Testing

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-019

Purpose

Define Model Testing within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Model Testing within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Model Testing within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Model Testing within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ModelTestingDefined
- ModelTestingRegistered
- ModelTestingStateChanged
- ModelTestingReviewRequested
- ModelTestingGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Model Testing has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.

book-03-10-20-model-roadmap.md

Book 03.10.20 - Model Roadmap

Version: v12 draft

Status: Draft

Document ID: MAL-V03-B03-10-020

Purpose

Define Model Roadmap within the Model Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Model Roadmap within the Model Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Model Roadmap within the Model Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Model Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Model Roadmap within the Model Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ModelRoadmapDefined
- ModelRoadmapRegistered
- ModelRoadmapStateChanged
- ModelRoadmapReviewRequested
- ModelRoadmapGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Model Roadmap has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B010
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v12 draft	2026-06-29	Draft	Initial Model Ecosystem blueprint content for Mariam Architecture Library v12.